

REST 練習 10 (延伸題) : JSON Array 批次處理與 BAPI 整合

Lecture

rs01~rs09 的每一個 **POST/PUT** 都只處理一筆資源：body 是一個 JSON 物件 (`{...}`)，對應到程式裡的一個 ABAP 結構 (`TYPE ty_xxx`)。但實務上很常遇到「呼叫端一次要送很多筆」的情境——例如旅行社系統一次幫十個客戶訂同一班機的位子，不會想為每一筆都各發一次 HTTP request (十次 TCP 連線、十次驗證、十次交易)。這種情境的 body 是一個 JSON 陣列 (`[{...}, {...}, ...]`)，伺服器要做的事情變成：把整個陣列反序列化成一張 **Internal Table**，逐筆處理，最後把每一筆的處理結果 (成功或失敗) 也組成一個陣列回傳。這題就是補這個模式：rs03 教過「Internal Table 序列化 JSON Array」(輸出方向)，這題教反方向——「JSON Array 反序列化 Internal Table」(輸入方向)。

這題另一個重點是呼叫 **BAPI**。rs06~rs09 寫入資料庫都是直接下 **INSERT/UPDATE/DELETE** Open SQL 語句——這在教學上很直接，但正式的 SAP 系統裡，跨模組、跨系統的資料寫入通常不會直接動 Open SQL，而是呼叫 **BAPI** (Business API，一種公開、穩定、有版本相容承諾的標準函式介面)。BAPI 內建了完整的業務規則檢查 (例如這題用的 **BAPI_FLBOOKING_CREATEFROMDATA** 會自己檢查航班、客戶是否存在，檢查失敗時透過標準的 **RETURN** 表格回傳結構化的錯誤訊息，不用像 rs06/rs09 那樣自己寫 **SELECT SINGLE** 驗證外鍵)，而且 BAPI 遵守一個重要慣例：**BAPI 本身不做 COMMIT WORK**，呼叫端處理完所有想做的異動之後，要自己呼叫 **BAPI_TRANSACTION_COMMIT** (或發現有問題時呼叫 **BAPI_TRANSACTION_ROLLBACK**) 才會真正落地——這讓呼叫端有機會「先呼叫多個 BAPI、確認整批都符合預期，最後才一次決定要不要真的送出」，這正是這題「陣列裡逐筆呼叫 BAPI、最後統一 COMMIT」設計的理論基礎。

這題選用的 **BAPI_FLBOOKING_CREATEFROMDATA** 剛好是 SAP 標準系統內建、專門給教學/展示用的訂位建立 BAPI (座落在 **SAPBC_IBF_SBOOK** 這個標準示範套件)，介面單純 (一次建立一筆訂位)，這題示範的「批次」概念是在我們自己的 **Service** 類別裡用 **LOOP** 包起來，不是 BAPI 本身支援一次吃多筆——這也是實務上很常見的整合模式：很多 BAPI 介面設計成「一次一筆」，呼叫端如果要批次處理，就是自己寫迴圈逐筆呼叫、逐筆收集結果。

這題的批次語意是「盡量做，個別回報成敗」，不是「全有全無」：陣列裡 10 筆訂位，就算第 3 筆因為客戶不存在而失敗，其餘 9 筆該成功的還是成功——這跟 rs06 單筆 **POST** 的「一次只認一個結果」不一樣，比較接近很多批次匯入 API 的實際行為 (想像上傳一個 Excel 匯入 100 筆資料，通常不會因為其中 1 筆錯誤就讓全部 100 筆都不處理)。回應是一個陣列，陣列裡每一筆都帶著自己的 **success/message**，呼叫端要自己檢查每一筆的結果，不能只看整個 HTTP 狀態碼判斷「全部都成功了嗎」。

學習目標

- 學會用 **/UI2/CL_JSON=>DESERIALIZE** 直接把一個 JSON 陣列 (不是物件) 反序列化進一個 **STANDARD TABLE** 型別的 Internal Table，跟 rs03 教過的「Internal Table 序列化 JSON Array」正好是反方向的操作
- 認識 **BAPI (Business API)** 的角色與慣例：穩定的公開介面、內建業務規則檢查、**RETURN** 表格回傳結構化訊息 (**TYPE/MESSAGE** 等欄位)、**本身不做 COMMIT WORK**，呼叫端要另外呼叫 **BAPI_TRANSACTION_COMMIT** 才會真正寫入資料庫
- 學會「陣列輸入、逐筆處理、陣列輸出」的批次處理模式：**LOOP AT** 輸入陣列，每一筆獨立呼叫 BAPI，獨立記錄成功/失敗，不因為其中一筆失敗就讓整批都失敗

- 分辨這題「部分成功」的回應語意 (200 OK + 陣列裡有成功也有失敗) 跟 rs06/rs09 「單筆 all-or-nothing」 (201/400/409 三選一) 的本質差異——批次 API 的「整體 HTTP 狀態碼」通常沒辦法完整表達「10 筆裡有 7 筆成功、3 筆失敗」這種混合結果，要看回應內容本身

事前準備

ADT 端物件已由課程準備好 (\$TMP)：

- ZCX_RS10_BATCH_ERROR——請求層級的例外 (例如陣列整個是空的)，不用來表達單筆訂位的成敗
- ZCL_RS10_BOOKING_BATCH_SERVICE——公開 ty_request/tt_request (輸入陣列的每一筆)、ty_result/tt_result (輸出陣列的每一筆結果)；validate_requests (純邏輯，附 ABAP Unit 測試)、create_bookings (呼叫 BAPI 的批次處理，不寫測試)
- ZCL_RS10_APP——Application Class，router 掛 /bookings/batch → ZCL_RS10_BATCH；覆寫 HANDLE_CSRF_TOKEN 跳過 CSRF 檢查
- ZCL_RS10_BATCH——薄層 Resource，只有 POST

題目需求 (對照已建好的答案物件)

1. 型別設計——輸入陣列的每一筆 ty_request、輸出陣列的每一筆 ty_result：

```
``abap TYPES: BEGIN OF ty_request, carrid TYPE s_carr_id, connid TYPE s_conn_id, fldate TYPE s_date, customid TYPE s_customer, class TYPE s_class, passname TYPE s_passname, counter TYPE s_countnum,"訂位一定要透過櫃台或旅行社成立，這裡固定走櫃台通路 END OF ty_request, tt_request TYPE STANDARD TABLE OF ty_request WITH EMPTY KEY,
```

```
BEGIN OF ty_result, index TYPE i,"對應輸入陣列的第幾筆 ( 0-based ) success TYPE abap_bool, carrid TYPE s_carr_id, connid TYPE s_conn_id, fldate TYPE s_date, bookid TYPE s_book_id,"失敗時維持初始值 message TYPE string, END OF ty_result, tt_result TYPE STANDARD TABLE OF ty_result WITH EMPTY KEY. ``
```

1. ZCL_RS10_BATCH~POST——反序列化直接進 **Internal Table**，這是這題跟前面九題最大的語法差異：

```
``abap DATA(lv_body) = io_entity->get_string_data( ).
```

```
DATA lt_requests TYPE zcl_rs10_booking_batch_service=>tt_request. /ui2/cl_json=>deserialize( EXPORTING json = lv_body pretty_name = /ui2/cl_json=>pretty_mode-camel_case CHANGING data = lt_requests ).
```

```
zcl_rs10_booking_batch_service=>validate_requests( lt_requests ).
```

```
DATA(lt_result) = zcl_rs10_booking_batch_service=>create_bookings( lt_requests ). ``
```

lt_requests 宣告成 TYPE zcl_rs10_booking_batch_service=>tt_request (一個 STANDARD TABLE)，DESERIALIZE 的 CHANGING data 參數餵給它，/UI2/CL_JSON 會自動判斷 body 是陣列還是物件，對應塞進表格或結構——不需要額外的參數告訴它「這次是陣列」。

1. create_bookings (批次呼叫 BAPI，核心邏輯)：

```
``abap METHOD create_bookings. DATA ls_booking_data TYPE bapisbonew. DATA lv_airlineid TYPE bapisbokey-airlineid. DATA lv_bookingid TYPE bapisbokey-bookingid. DATA ls_price TYPE bapisbopri. DATA lt_return TYPE STANDARD TABLE OF bapiret2.
```

```
LOOP AT lt_requests INTO DATA(ls_request). DATA(lv_index) = sy-tabix - 1.
```

CLEAR: ls_booking_data, lv_airlineid, lv_bookingid, ls_price, lt_return.

ls_booking_data-airlineid = ls_request-carrid. ls_booking_data-connectid = ls_request-connid.

ls_booking_data-flightdate = ls_request-fldate. ls_booking_data-customerid = ls_request-customerid.

ls_booking_data-class = ls_request-class. ls_booking_data-passname = ls_request-passname.

- 業務規則：訂位一定要透過「櫃台」或「旅行社」其中一種通路成立，兩者都不填 BAPI 會拒絕

ls_booking_data-counter = ls_request-counter.

CALL FUNCTION 'BAPI_FLBOOKING_CREATEFROMDATA' EXPORTING booking_data = ls_booking_data
IMPORTING airlineid = lv_airlineid bookingnumber = lv_bookingid ticket_price = ls_price TABLES return =
lt_return.

DATA(lv_error_message) = ''. LOOP AT lt_return INTO DATA(ls_msg) WHERE type = 'E' OR type = 'A'.

lv_error_message = ls_msg-message. EXIT. ENDLOOP.

IF lv_error_message IS NOT INITIAL. APPEND VALUE #(index = lv_index success = abap_false carrid =
ls_request-carrid connid = ls_request-connid fldate = ls_request-fldate message = lv_error_message) TO
rt_result. ELSE. APPEND VALUE #(index = lv_index success = abap_true carrid = ls_request-carrid connid =
ls_request-connid fldate = ls_request-fldate bookid = lv_bookingid message = |訂位建立成功|) TO rt_result.
ENDIF. ENDLOOP.

CALL FUNCTION 'BAPI_TRANSACTION_COMMIT' EXPORTING wait = abap_true. ENDMETHOD. ``

幾個實作細節：

- **BAPIRET2** (**RETURN** 表格的標準結構) 用 **TYPE** ('E' 錯誤 / 'A' 中斷 / 'S' 成功 / 'W' 警告 / 'I' 訊息) 判斷這則訊息的嚴重程度——這題只把 'E' / 'A' 當作「這筆訂位失敗」，'W'/'I' 這類非阻斷性訊息忽略不管 (BAPI 有時候會回一些提示性訊息，不代表操作失敗)
- **CLEAR** 每一圈都要做：CALL FUNCTION 的 IMPORTING/TABLES 參數如果上一輪呼叫有值、這一輪 BAPI 因為某種原因沒有覆寫，殘留的舊值可能被誤判成這一輪的結果——這是呼叫函式模組時的通用地雷，不是這個 BAPI 特有的
- **LOOP AT it_requests** 每一筆都獨立呼叫、獨立記錄結果，中間沒有 IF 判斷「前面失敗了就跳過後面」——這正是「盡量做，個別回報成敗」的實作方式，第 3 筆失敗完全不影響第 4 筆繼續嘗試
- **BAPI_TRANSACTION_COMMIT** 只呼叫一次，在整個迴圈跑完之後——不是每呼叫一次 **BAPI_FLBOOKING_CREATEFROMDATA** 就 COMMIT 一次；這個設計讓「10 筆裡 7 筆成功」的情境下，7 筆成功的訂位在最後這一次 COMMIT 裡一起真正寫入資料庫，3 筆失敗的 (BAPI 內部檢查沒過) 本來就沒有寫入任何東西，不需要特別 ROLLBACK

SICF 掛載步驟

沿用既有的 /sap/bc/zrest_training 分類節點，這題新增子節點：

1. SICF 交易碼，在 zrest_training node 上右鍵 → **New Sub-Element**，Service Name 填 **rs10**
2. Handler List 掛 **ZCL_RS10_APP**
3. Activate Service
4. 用 **SPROX_HTTP_REQUEST** 測試 (**URL** 一律填完整網址)：POST
https://erpdemo01.itts.com.tw:44300/sap/bc/zrest_training/rs10/bookings/batch?sap-

client=130 · Req. Body (混合一筆會成功、一筆會失敗的示範，客戶 id 先用 SE16 查一筆 SCUSTOM 真實存在的 ID)：

```
`json [
{"carrid":"LH","connid":400,"fldate":"20270101","customid":"00000001","class":"Y","passname":"Mickey Mouse","counter":"00000001"},
{"carrid":"ZZ","connid":400,"fldate":"20270101","customid":"00000001","class":"Y","passname":"Donald Duck","counter":"00000001"} ] `
```

- **fldate** 一定要用未來日期：這個系統的 SFLIGHT 訓練資料整張表最新只到 2019-10-19，BAPI_FLBOOKING_CREATEFROMDATA 會檢查航班日期不能是過去，用歷史日期一定會被拒絕 (Flight date ... is in the past)；carrid/connid/fldate 這組鍵值如果資料庫裡還沒有，要先用 rs06 的 POST /flights 建一筆 (航線 connid 沿用既有的即可，只是換一個未來 fldate)
- **counter** (櫃台代碼) 是必填：BAPISBONEW 結構的業務規則是「訂位一定要透過櫃台或旅行社其中一種通路成立」，兩者都不填會被 BAPI 拒絕 (No travel agency or counter passed)；00000001 是這系統裡 LH 確認存在的櫃台代碼 (SCOUNTER 查證)，若要用其他航空公司要另外查一組有效的 counter
- 確認回應是 200 · Body 是一個陣列，第一筆 success:true 帶著新的 bookid，第二筆 success:false (carrid=ZZ 不是合法航空公司) 帶著 BAPI 給的錯誤訊息
- 用 SE16/rs09 的 GET 端點驗證第一筆真的寫進 SBOOK 了、第二筆沒有
- 送一個空陣列 []，確認回 400 + { "errorCode":"VALIDATION_FAILED", ... }

預期輸出 (範例)

POST /bookings/batch (一筆成功、一筆失敗)：HTTP 狀態碼 200

```
[
  {"index":0,"success":true,"carrid":"LH","connid":400,"fldate":"2027-01-01","bookid":<bookid>,"message":"訂位建立成功"},
  {"index":1,"success":false,"carrid":"ZZ","connid":400,"fldate":"2027-01-01","bookid":0,"message":"<BAPI 回傳的錯誤訊息文字>"}
]
```

空陣列：HTTP 狀態碼 400

```
{ "errorCode": "VALIDATION_FAILED", "message": "至少要有一筆訂位資料" }
```

團隊實務備註

- 這題是唯一一個「呼叫 BAPI 而不是直接下 Open SQL」的練習——rs06/rs07/rs08/rs09 都是直接 INSERT/UPDATE/DELETE，這題刻意示範另一條路：正式系統做跨模組整合時，優先看有沒有現成 BAPI 可以呼叫，而不是直接動別的模組負責的資料表；BAPI 帶有業務規則檢查跟版本相容承諾，直接 Open SQL 沒有這些保障
- BAPI_TRANSACTION_COMMIT 只在最外層呼叫一次是這題最重要的紀律：如果在 LOOP 裡面每筆都呼叫一次 COMMIT，會失去「先跑完整批、確認狀況、再決定要不要送出」的彈性 (雖然這題目前的設計是不管

前面結果如何都繼續跑完整批，但只在最後統一 COMMIT 的寫法，為將來想加上「超過 N 筆失敗就整批 ROLLBACK」這類規則留了修改空間)

- BAPIRET2 的 MESSAGE 欄位是組好的完整訊息文字 (不需要再查訊息類別/訊息號自己組字串)，這是 BAPI 慣例的好處之一——呼叫端不用知道訊息背後的 T100 訊息類別結構，直接顯示 MESSAGE 欄位就是一句完整、人類看得懂的話
- 這題沒有像 rs06/rs09 那樣自己寫 SELECT SINGLE ... FROM sflight/scustom 驗證外鍵——因為 BAPI 自己會做這些檢查，這正是呼叫 BAPI 比自己重寫驗證邏輯的好處，不用重複造輪子
- 這題最早版本沒有 counter 欄位，SICF 實測時才發現 BAPISBONEW 有「訂位必須透過櫃台或旅行社」的隱性業務規則 (BAPI 回 No travel agency or counter passed)——這正好示範了 BAPI 業務規則檢查「藏」在介面背後、光看 IMPORTING/TABLES 簽章不一定看得出所有必填的隱性條件，只有實際呼叫過 (或查閱 SCOUNTER/STRAVELAG 這類關聯 customizing 表) 才會發現；這也是為什麼 rs11 特別把 BAPI 的 TESTRUN 機制暴露成 API 參數——先小規模測試、看 RETURN 訊息，是面對這類複雜 BAPI 介面最務實的除錯方式，比光看程式碼推敲更可靠
- 這題的 POST body 刻意直接用陣列當根節點 ([{...}, {...}])，沒有額外包一層 {"data": [...]} 或 {"REQUEST": [...]} 這種 wrapper key——這是刻意的設計選擇，不是隨便省略：HTTP Method (POST) 本身就已經表達了「這是一個請求」，body 裡再包一層通用命名的外層 key (尤其是像 REQUEST 這種完全沒有業務語意的名稱) 對前端/呼叫端沒有實質幫助，只是多一層要拆的殼；只有當這個 endpoint 除了這批陣列資料之外，還需要傳遞跟陣列平行的「全域性」參數 (例如呼叫者身分 operator_id、來源系統代碼) 時，才有必要在外層包一個物件、並把陣列放在一個有業務意義的 key 底下 (例如 "bookings": [...], 而不是通用的 "data"/"REQUEST")——這題目前不需要這類全域參數，所以維持陣列直接當根節點

思考題

1. 如果呼叫端想要「這個批次只要有任何一筆失敗，全部都不要真的送出」(比較接近 rs09 期末整合時 ex23 教過的 LUW all-or-nothing 語意)，這題的 create_bookings 該怎麼改？(提示：把 BAPI_TRANSACTION_COMMIT 換成有條件的 COMMIT/ROLLBACK，要在迴圈跑完之後、根據 rt_result 裡有沒有 success = abap_false 的項目來決定)
2. 這題的批次大小 (陣列筆數) 完全沒有上限——如果呼叫端一次送 10,000 筆訂位，這支 API 可能會遇到什麼問題？(提示：想想 HTTP 逾時、RETURN 表格增長、單一 Dialog Work Process 執行時間限制)
3. validate_requests 只檢查「陣列不能是空的」，沒有檢查陣列裡每一筆的 carrid/customid/class/passname 格式——如果想在呼叫 BAPI 之前就先做格式檢查 (不用等 BAPI 執行到一半才發現格式錯誤)，這段邏輯該加在哪裡？跟 rs09 validate_booking_fields 的角色有什麼可以共用的地方？

答案

見 zcx_rs10_batch_error.clas.abap、zcl_rs10_booking_batch_service.clas.abap (+ .testclasses.abap)、zcl_rs10_app.clas.abap、zcl_rs10_batch.clas.abap (SAP 端物件 ZCX_RS10_BATCH_ERROR / ZCL_RS10_BOOKING_BATCH_SERVICE / ZCL_RS10_APP / ZCL_RS10_BATCH)。SICF Service 路徑 /sap/bc/zrest_training/rs10。